Equipe: Felipe Bernardes, Carlos Eduardo e Vinicius Cavalcanti
Turma: Estrutura de Dados 2026.2

Baseando-se na experiência do integrante do grupo, Carlos, por ter conhecimento em diferentes linguagens de programação, optamos por implementar as funções de ordenação por C++ , já o pré-processamento e a implementação dos gráficos, por Python, onde o primeiro grupo, destinado às funções Insertion Sort, Selection Sort e Bubble Sort, foi responsável por Carlos, junto a base de implementação que seguiríamos para usar cada escolha de função do primeiro e segundo grupo.

Na base de complemento de cada algoritmo foi usado a função de primeiramente ler as palavras do arquivo pré-processado, no qual seria usado para criar uma lista para uso em cada algoritmo, logo o tempo de execução é retornado para o Python, para ser usado construção dos gráficos, criados por Carlos.

Na aplicação principal foi utilizado o pré-processamento das palavras, a fim de remover elementos desnecessários ou que atrapalhariam a ordenação das palavras, onde, em cada documento de palavras, do original para o pré-processamento ocorreu a mudança de:

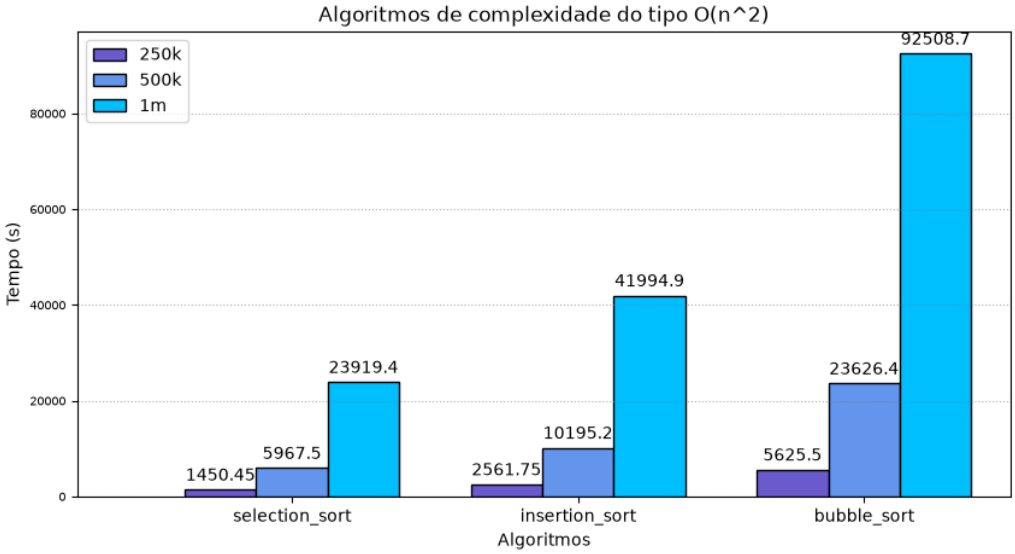
- 250.000 palavras -> 251.346 palavras
- 500.000 palavras -> 502.688 palavras
- 1.000.000 palavras -> 1.005.378 palavras

Ainda falando da parte feita por Carlos, para o Bubble Sort, ele utilizou a estratégia de ordenação por trocas adjacentes onde a variável “ultimo” define o limite da parte do vetor que ainda precisa ser percorrida, e onde irá parar naquela passagem e então usando um for para percorrer o vetor, comparando cada palavra com a seguinte. Se a palavra atual for alfabeticamente maior que a próxima, as duas são trocadas. Ao final de cada passagem, a maior palavra da parte não ordenada chega à sua posição definitiva e portanto limite “ultimo” é diminuído, evitando comparações desnecessárias. O processo é repetido até que uma passagem inteira ocorra sem nenhuma troca, ou seja seguindo a regra de sempre comparar elementos em posições consecutivas.

No Selection Sort foi usado ordenação por seleção direta onde o foco é o menor elemento da parte não ordenada, que se usa a variável “inicio” que é o contrário usado anteriormente no Bubble Sort, sempre percorrendo a lista achando o menor elemento e então o colocando no início, ao mesmo tempo que atualiza o início para a próxima posição dele, usando como auxílio o “pos_menor” e “temp” para identificar o índice das menores palavras encontradas e também para armazenar uma palavra temporariamente para troca.

Já no Insertion Sort, se usa a ordenação por inserção no qual se constrói uma sequência ordenada, inserindo cada novo elemento na posição correta em relação aos elementos que já foram organizados porém em vez de ordenar diretamente o vetor original, optou-se utilizar um vetor auxiliar chamado “copia”, que vai sendo preenchido em ordem crescente com o mesmo tamanho do vetor original, e com a ajuda do “quant_copia” que controla quantos elementos já foram inseridos. O primeiro elemento é colocado diretamente na primeira posição de cópia. Tendo em vista a base inicial, para cada palavra seguinte, o algoritmo percorre a parte ordenada de “copia”, de trás para frente, enquanto os elementos maiores que a palavra atual são deslocados uma posição para a direita e então, ao encontrar a posição correta, se insere a palavra no início, no meio ou no final da sequência ordenada, repetindo o processo até que todas as palavras tenham sido inseridas.

Utilizando a geração de gráficos Matplot em Python foi mostrado o resultado do tempo gasto pelo primeiro grupo como:



Terminado o primeiro grupo, o integrante Felipe, ficou responsável pela implementação dos primeiros 2 algoritmos do 2° grupo, que seria o Shellsort e Heapsort, assim como ficou também pela criação do Relatório utilizado agora.

Primeiramente, no Shellsort, foi utilizado a ordenação por incrementos decrescentes que foi optada por usar a uma sequência de gaps conhecida como sequência de Ciura, com valores estendidos para intervalos maiores, usando os mesmos empiricamente apenas até 701, para vetores maiores que demandam lacunas superiores, os valores subsequentes são calculados multiplicando recursivamente o último termo por um fator de aproximadamente 2,25. Para cada gap, o algoritmo percorre o vetor a partir da posição desse intervalo e cada palavra é comparada com elementos anteriores separados pelo gap, já os elementos maiores são deslocados para a direita, até que a palavra atual encontre sua posição adequada dentro daquele intervalo. E ,portanto, o processo é repetido com intervalos progressivamente menores, até chegar ao gap igual a 1, que realiza uma ordenação semelhante ao Insertion Sort tradicional que segue a regra de comparar e movimentar elementos que estão distantes uns dos outros.

No Heapsort foi usado a construção do Max-Heap no qual cada nó pai possui um valor maior ou igual ao dos seus filhos, seguindo o objetivo inicial do Heap de colocar o maior elemento na raiz da árvore, removê-lo da parte não ordenada e reorganizar o Heap até que todos os elementos estejam ordenados. O algoritmo constrói um Max-Heap, organizando as palavras para que a maior fique na primeira posição dando ele para a função “heapify”, ele verifica se a raiz de uma subárvore é menor que algum de seus filhos e, caso seja, troca a raiz com o maior filho e continua ajustando a subárvore. Após construir o Heap, a maior palavra é trocada com o último elemento ainda não ordenado e o tamanho considerado do Heap é reduzido, excluindo a palavra já posicionada, por fim, a função heapify é chamada novamente para restaurar a propriedade do Max-Heap.O processo se repete, então, até que todas as palavras estejam em ordem crescente

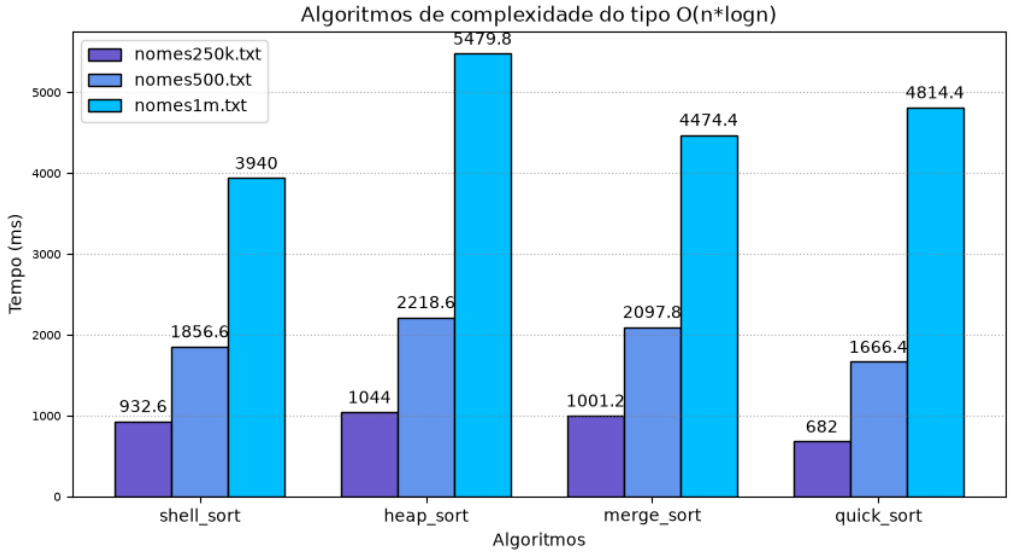
Já os outros 2 algoritmos do 2° grupo foram feitos pelo membro Vinicius, onde foi implementado o Mergesort e Quicksort.

Para o Mergesort, que utiliza a estratégia de dividir para conquistar, dividindo o vetor em duas metades sucessivamente até que cada parte tenha apenas um elemento, foi usado o “merge_sort_recursivo” que divide o vetor em partes menores e chama a si mesma para ordenar cada metade e então o “merge” que compara os elementos das duas metades já ordenadas e os intercala no vetor original, formando uma sequência ordenada, e

somente no “merge_sort” é que inicia a ordenação do vetor. No entanto, mesmo que mantenha um desempenho previsível, ele fica utilizando memória extra.

Já para o Quicksort, que também utiliza a estratégia de dividir para conquistar, mas, em vez de intercalar as metades, ele escolhe um pivô para separar os elementos menores ou iguais dos maiores. Onde se inicia primeiramente escolhendo o pivô comparando o primeiro, o elemento do meio e o último elemento do trecho e chega a parte de reorganizar o vetor, colocando os elementos menores ou iguais ao pivô à esquerda e os maiores à direita, deixando o pivô em sua posição definitiva, logo então, se aplica o processo recursivamente às partes à esquerda e à direita do pivô, ficando assim pronto para iniciar a sua ordenação.

Juntando os algoritmos dos 2° grupo, se deu os gráficos de tempo:



E por fim o Relatório foi feito por Felipe Bernardes, o Vídeo por Carlos e a base do processo de Ordenação por Carlos, onde cada um dos 3 participantes contribuiu com os algoritmos de forma igual.